

Measured, Not Asked: Free Saliency Signals for Consolidation and Routing at Small Scale

Anonymous authors

Paper under double-blind review

Abstract

An agent that wants to move knowledge from its context into its weights, or to read only part of a long context, needs some signal of what matters. Three such signals are available without a reward model or a judge: the keep/drop decisions a context compactor already makes, the divergence between a model’s predictions with and without a document in context, and the region a model says it needs to read. We test all three on Qwen2.5 models from 0.5B to 7B, reporting every signal-strength figure next to a matched control. Two findings come out of this. The first is that signals the model is asked to state are much weaker than signals measured from it. Asked to list the spans worth keeping, the 0.5B and 1.5B models return the prefix 0, 1, 2, ... on every event, and asked which of eight regions holds an answer, the 1.5B model does no better than always naming the same region. Measured from the same models, the signals work: the keep decision read off the logits clears its control on HotpotQA at both sizes ($+3.75\sigma$ and $+4.77\sigma$ over chance), and scoring each region by the log-odds of a relevance question reaches a hit rate of 0.33 to 0.42 against a chance rate of 0.125. This logit read beats the stated choice at all three scales (by 0.19 at 7B, $t(2)=24$) and beats late-layer attention probing at 0.5B and 7B. The second finding is that a strong signal can still be a poor training target. Consolidating on the compactor’s keep decisions, which carry a $+15.85\sigma$ saliency signal, recovers fewer evicted facts than consolidating on uniformly sampled spans in all five synthetic corpora (1.7% vs 7.6%, paired $t(4)=3.10$), because the compactor never keeps some fact types. Gating distillation on the context gap does no better than a random token subset of the same size (5 seeds, $t(4)=0.74$). Neither signal beats uncured coverage. Along the way, matched controls exposed eight artifacts in our own pipeline, six of which had inflated a result in our favour.

1 Introduction

A long-running agent accumulates context faster than it can keep it. Context is expensive to attend over and disappears when it is compacted or truncated; weights are cheap to read and persist. So there are two obvious things to want: move what matters from the context into the weights, and read only the part of the context that a given question needs. The mechanics of both are routine by now (a LoRA update in one case, a sparse KV-cache read in the other). The hard part is knowing what matters.

Most prior work pays for that knowledge. Nightly consolidation writes reflections with a second generation pass (Dennis et al., 2026), SCoL learns where to write with meta-RL (Wang et al., 2026), and CompactionRL trains the compactor itself with reinforcement learning (Li et al., 2026). We asked what an agent already has for free, and found three candidates.

- **The compaction decision.** Each time an agent compacts its context, it decides which spans to keep verbatim. That decision labels some spans as more important than others, at no extra cost.
- **The context gap.** If a frozen model is run on the same continuation twice, once with a skill document in its context and once without, the per-token KL divergence between the two runs measures how much the document changed its predictions. This takes two forward passes.

- **The attention declaration.** Declarative Attention (Ho et al., 2026) asks a model to state which region of its context it needs, then skips the rest of the KV-cache read. The statement is a salience label too.

We tested each one as a training or routing signal on models that fit on a single GPU, between 0.5B and 7B parameters. The results fall along two axes. One is how the signal is obtained: *measured* from the model’s logits or activations, or *asked* for as generated text. This largely decides whether a usable signal exists at small scale. The other is whether a signal that does exist is worth training on. This decides whether it helps.

We present the work as a measurement study. The methods are hypotheses, and two of the three consolidation hypotheses turned out to be false. Our contributions are:

1. Evidence from two mechanisms that, at 0.5B to 7B, signals a model is asked to state are weak or absent while the same model’s measured signals are strong (§4), with a third signal, the context gap, that is measured by construction (§5.2).
2. Evidence that two free signals which clear matched controls still do not beat uniform coverage as training targets, for two different reasons (§5).
3. A logit-read substitute for declarative attention at scales where declaring does not work, together with a content-shuffle control that separates routing by content from routing by position (§4.2).
4. A catalogue of eight artifacts that matched controls caught in our own pipeline, as a case for reporting a control beside every signal-strength number (§6).

2 Related work

Consolidation from context. Beyond Inference-Only Deployment (Dennis et al., 2026) shows that periodic LoRA consolidation beats cascading compaction, and that median per-token cross-entropy tracks judged accuracy far better than the mean ($r = +0.99$ against -0.51). Its training targets are reflected facts produced by an extra generation pass. We train on the compactor’s own decisions instead, which removes that pass and takes reflection quality out of the comparison; their recipe serves as one of our baselines. SCoL (Wang et al., 2026) learns where in the network to write consolidated knowledge, which is complementary to our question of what to write. What to Keep, What to Forget (Colaco & Lahjouji, 2026) treats compaction as a rate-distortion problem and stops at the compactor, whereas we use the compactor’s output as training signal. CompactionRL (Li et al., 2026) trains the compactor with RL while we keep it frozen and train the backbone, so the two could be combined. Compression-Aware Abstention (Khodabandehlou & Krishnamachari, 2026) trains a model on KV-compression masks to refuse when compression has removed the evidence for an answer; we reproduce its direction at 0.5B in §5.1.

Gated distillation and token selection. Skill-to-LoRA (Zhang & Qi, 2026) distills each skill into its own adapter with uniform self-distillation and is the baseline for §5.2. ThinkSwitch (Saini & Pandey, 2026) distills context into LoRA adapters with weight interpolation, also without gating. PEAM (Guo et al., 2026) builds an embodied agent’s experience into its parameters by contrasting trajectories, which is the closest work to our gate, though it operates on different objects with a different signal. Titans (Behrouz et al., 2024) treats the gradient of an associative-memory loss as a surprise signal at test time. Our context gap is a surprise signal in a similar spirit, but computed as a divergence between two forward passes and used offline. Two recent results point the other way from our §5.2 finding. Rho-1 (Lin et al., 2024) reports that restricting the loss to high-scoring tokens helps pretraining, and TIP (Xu et al., 2026) finds that training on fewer than 10% of tokens, selected by teacher–student divergence and entropy, nearly matches full-token on-policy distillation. We compare the gate against a random subset under identical masking, which separates the effect of the ranking from the effect of training on a subset at all. In our setting the subset helps and the ranking does not. We do not know whether the same split holds in theirs.

Attention routing. Declarative Attention (Ho et al., 2026) has a 27B or 31B model emit `<global>`, `<focus>` or `<local>` in its chain of thought and skips most of the KV-cache read to match. It attends to 52.0% and 31.1% fewer tokens at a cost of 1.27 and 2.75 accuracy points, zero-shot, and notes that the accuracy cost shrinks as models grow. We test the other end of that trend. Sentinel (Zhang et al., 2025) and

work on retrieval heads (Wu et al., 2024) route by probing attention rather than asking, and we run that approach on the same models, layouts and probes as the others.

3 Setup and measurement protocol

Models and compute. We use Qwen2.5-Instruct (Qwen Team, 2024) at 0.5B, 1.5B and 7B, and LoRA (Hu et al., 2021) with rank 16 and $\alpha = 32$ on every attention and MLP projection. The 0.5B and 1.5B models run in fp16. The 7B model runs in 4-bit nf4 quantisation so that it fits on a 16 GB T4, the only hardware used.

Salience lift. For a signal that marks spans, lift is the rate at which fact spans are kept divided by the rate at which filler spans are kept. We always report it next to a positional control, the lift obtained by simply keeping the first N spans, together with its margin in σ over chance and over that control.

Statistics. We compare single runs with a binomial σ and multi-seed runs with a paired t-test across seeds, judged against the two-sided 5% critical value for $k-1$ degrees of freedom: 4.30 for three seeds and 2.78 for five. We call a difference significant only if it clears that value, and otherwise report it as a tie.

Content shuffle. For routing, a hit rate alone cannot tell whether a model follows content or favours a position. So we move the content to different regions and ask again. **content_dependence** is the hit rate after shuffling minus chance, and **slot_stable_rate** is the fraction of probes on which the choice stays in the same slot even though the content moved. Values near $1/K$ indicate routing by content and values near 1 routing by position. A best-constant control, which always names the most common gold region, catches a model that names the same region regardless.

4 Finding 1: measured beats asked

4.1 The compaction decision

The compactor splits the evictable prefix of a trajectory into sentence spans and keeps $\text{ceil}(\rho \cdot n)$ of them, with $\rho = 0.25$. We obtain its keep decision in two ways. In **generate** mode the model writes out the indices of the spans to keep. In **read** mode we ask it about each span in turn and score $\log p(\text{Yes}) - \log p(\text{No})$ from the logits, keeping the top $\rho \cdot n$. We shuffle the spans before showing them, so position cannot pass for salience.

When asked to list spans, both Qwen models answered with the prefix 0, 1, 2, ... on 100% of events, whatever the content. Because the spans were shuffled, this is exactly random selection. The logit read gives a real signal, but how strong depends on the data:

Backbone	Data	Fact keep	Lift	Pos. control	vs chance	Verdict
Qwen2.5-1.5B	HotpotQA	0.337	1.34x	0.45x	+3.75 σ	clears
Qwen2.5-0.5B	HotpotQA	0.360	1.44x	0.45x	+4.77 σ	clears
Qwen2.5-1.5B	synthetic, marked	0.646	2.72x	1.66x	+15.17 σ	clears
Qwen2.5-0.5B	synthetic, marked	0.097	0.37x	1.66x	-6.18 σ	below chance
Qwen2.5-1.5B	synthetic, unmarked	0.344	1.37x	1.71x	+3.47 σ	below control

Table 1: salience of the logit-read keep decision, each row at least 288 fact spans.

HotpotQA is the representative case. There is no marker phrase or shared vocabulary to exploit, and both models keep fact spans well above chance. We quote the margin over chance there, because the positional control falls below chance on that benchmark (the compactor prepends carried-over filler to each span list), which would make the margin over control look better than it is. The synthetic benchmark plants each fact with a marker phrase, and the 1.5B model’s very large lift there comes largely from detecting that marker: with the marker removed, it still beats chance but loses to the positional control. The 0.5B model is the

best compactor on HotpotQA and the worst on marked synthetic data, so no ordering of models holds across datasets.

The consolidation experiments in §5.1 use the marked synthetic benchmark, where the corrected run (48 trajectories, 97 compaction events, 6,244 spans) keeps fact spans at 0.663 and filler at 0.236: a 2.81x lift, $+15.85\sigma$ over chance and $+8.40\sigma$ over the positional control, with no fallbacks and no empty keeps. This makes it a deliberately favourable case for the gate: if consolidating on the compactor’s decisions helps anywhere, it should help where those decisions separate facts from filler this cleanly. The signal is uneven across fact types, though. `db_engine`, `owner` and `version_pin` are kept in 100% of events, `config_flag` in 3.4% and `auth_header` never. This unevenness turns out to matter in §5.1.

Measuring the signal is therefore necessary but not sufficient: the same logit read that works for the 0.5B model on HotpotQA falls below chance on marked synthetic data, so whether a model carries a usable signal has to be checked for each model and dataset.

4.2 The attention declaration

Task. We take HotpotQA trajectories, scatter their supporting paragraphs through the context, and split the context into $K = 8$ labelled regions. The model receives a question whose answer lies in exactly one region. Rendered contexts reach 8,966 tokens, well within the 32,768-token window of every model. There are 384 probes, and we permute the gold region for each. Three seeds resample the layouts and the shuffle over a fixed probe set, and we pin the gold distribution of each seed so that every model sees the same layouts.

Elicitations.

- **generate:** the model answers `FOCUS: <k>`, which we parse like a tool call, following Ho et al. (2026).
- **read:** for each region we ask whether it contains the answer and score $\log p(\text{Yes}) - \log p(\text{No})$, taking the highest-scoring region. This needs K batched prompts.
- **attention:** nothing is asked. We run the model on the full context and question and add up the attention that the final query position places on each region, across all layers. **attention_late** adds up only the second half of the layers. This is one forward pass over roughly the same number of tokens as **read**.

Table 2 gives the results and Figure 1 plots them. Two caveats apply to reading across scales. The seeds resample layouts over one fixed set of 384 probes, so the paired tests speak to variation over layouts rather than over probes. And we quantise the 7B model to 4 bits while the others run in fp16, so a change between 1.5B and 7B mixes scale with quantisation.

model	elicitation	hit	σ over constant	content dependence	slot stable	unparsed
0.5B	generate	0.193 ± 0.012	2.2	0.085 ± 0.021	0.281	0.019
0.5B	attention	0.151 ± 0.003	0.2	0.014 ± 0.012	0.882	0.000
0.5B	attention_late	0.234 ± 0.011	4.0	0.113 ± 0.029	0.356	0.000
0.5B	read	0.332 ± 0.004	7.7	0.203 ± 0.003	0.136	0.000
1.5B	generate	0.150 ± 0.008	0.1	0.030 ± 0.023	0.413	0.170
1.5B	attention	0.286 ± 0.023	6.0	0.163 ± 0.010	0.447	0.000
1.5B	attention_late	0.374 ± 0.017	9.2	0.258 ± 0.028	0.200	0.000
1.5B	read	0.378 ± 0.005	9.3	0.241 ± 0.005	0.136	0.000
7B	generate	0.230 ± 0.010	3.8	0.116 ± 0.020	0.260	0.000
7B	attention	0.229 ± 0.009	3.8	0.085 ± 0.018	0.595	0.000
7B	attention_late	0.359 ± 0.017	8.6	0.223 ± 0.019	0.242	0.000
7B	read	0.418 ± 0.011	10.7	0.285 ± 0.004	0.135	0.000

Table 2: routing on 384 HotpotQA probes, mean $\pm$ sd over three seeds. Chance is 0.125 and the best-constant control 0.148.

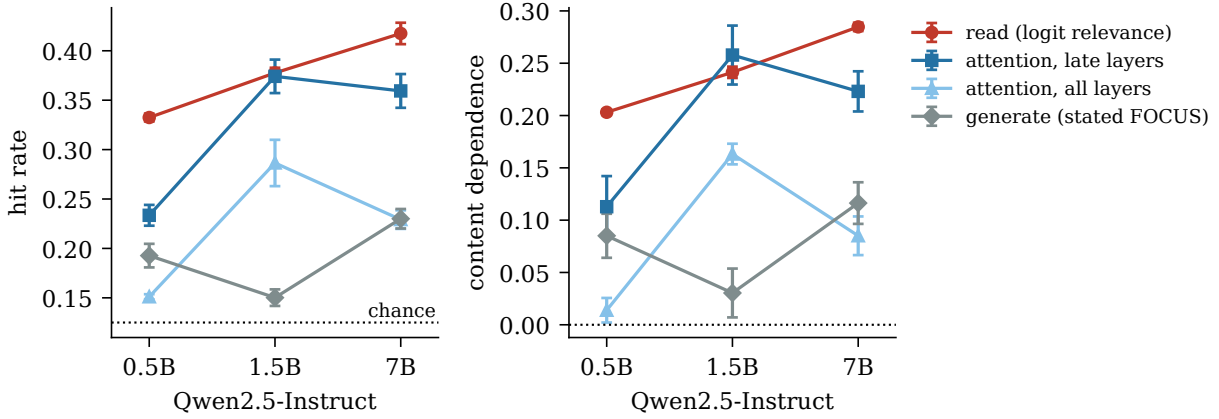


Figure 1: Routing accuracy by scale. Left: hit rate, with chance dotted. Right: content dependence, the hit rate after the content is moved to other regions, minus chance. Error bars are the standard deviation over three seeds. The logit read is the best or tied-best route at every scale; the stated declaration collapses at 1.5B and partly recovers at 7B.

The stated declaration stays well below the measured routes at every scale. At 0.5B its hit rate clears the constant control ($+2.2\sigma$), but the shuffle shows that this is mostly position: its content dependence is 0.085, less than half that of **read**. At 1.5B it cannot be told apart from always naming one region ($+0.1\sigma$). The model also refuses the **FOCUS** format on 17% of probes, and its slot-stable rate rises to 0.41. At 7B the refusals stop and the stated choice clears the constant control again ($+3.8\sigma$, and between $+3.7\sigma$ and $+4.0\sigma$ on each seed), with content dependence 0.116, which resembles 0.5B more than 1.5B. **read** still beats it by 0.188 in hit rate ($t(2) = 24.0$) and by 0.168 in content dependence ($t(2) = 12.3$). So the decline from 0.5B to 1.5B is not a trend that continues to 7B, but the gap between stated and measured routing persists at every scale we tested.

Between the two measured routes, **read** wins at 0.5B and 7B and ties at 1.5B. At 0.5B it leads the better attention variant in hit rate (0.332 vs 0.234, $t(2) = 24.9$) and in content dependence (0.203 vs 0.113, $t(2) = 4.91$). At 1.5B the two are level, 0.378 vs 0.374 ($t(2) = 0.30$) and 0.241 vs 0.258 ($t(2) = -0.85$). At 7B **read** leads again, 0.418 vs 0.359 ($t(2) = 12.0$) and 0.285 vs 0.223 ($t(2) = 4.69$). Its content dependence rises at each step (0.203, 0.241, 0.285), while that of **attention_late** rises to 1.5B and then falls (0.113, 0.258, 0.223). An earlier single-seed run had suggested that attention overtook **read** at 1.5B; with three seeds that difference disappears into layout noise.

Probing only works from the late layers. Summed over all layers, attention is useless at 0.5B, with content dependence 0.014 and the same slot chosen on 88% of probes. Restricted to the second half of the layers, it clears its control at every scale. The early layers attend by position and swamp the sum: one layer-0 query-key product in Qwen2.5-1.5B reaches 152,967, while late-layer peaks sit near 300. Without this ablation, a probing baseline would understate itself by more than an order of magnitude.

4.3 What the two mechanisms have in common

In both cases the model’s measured judgment is much better than what it says when asked. At 1.5B the generated **FOCUS** carries no content signal (0.030), while the same model’s logit read carries 0.241 and its late-layer attention 0.258. The small models do seem to have the relevant judgment. What they lack is the ability to express it reliably as structured output on demand, and going up to 7B closes only part of that gap. There is also no single right way to measure. The self-query and attention probing alternate between winning and tying across scales, and attention works only from the late layers. Of the routes we tried, the self-query was never worse than the alternatives, so it is the one we would use by default. The context gap in §5.2 is a measurement by construction and involves no elicitation at all.

5 Finding 2: salient is not trainable

5.1 Consolidating on the compaction decision

Method. After every fourth compaction event, a sleep phase fine-tunes a LoRA adapter on the spans the compactor kept, with a reservoir replay buffer carried across phases. The compactor is Qwen2.5-1.5B using the logit read; the model being consolidated and evaluated is Qwen2.5-0.5B.

Benchmark. Each synthetic trajectory is an agent conversation of 120 turns. We plant six facts (`auth_header`, `db_engine`, and so on) in the first twelve turns and fill the rest with filler text, so compaction evicts some of them. Each fact has one probe, which gives 288 probes over 48 trajectories per corpus. A probe counts as correct if the normalised gold answer, or an alias, appears in the model’s output. It counts as *evicted* if the answer no longer appears in the compacted context; these are the only probes on which consolidation can make a difference. Every question names a project unique to its trajectory, for reasons explained in §6. As in Dennis et al. (2026), the agent consolidates its own history: sleep phases train on the evaluation trajectories’ compaction events, and validation uses held-out training trajectories.

Arms. Besides our compaction-supervised adapter, we run (a) uniform replay, with the same sleep phases and budget but spans sampled uniformly from kept and dropped alike; (b) reflection, following Dennis et al. (2026); (c) cascading compaction with no weight update; and (d) the full context as a ceiling.

method (corpus 0, 114 evicted)	all-probe	evicted
no context (floor)	0.000	0.000
(c) cascading, no adapter	0.413	0.000
ours: compaction-supervised	0.451	0.000
ours + mask head	0.323	0.009
(b) reflection	0.389	0.000
(a) uniform replay	0.465	0.123
(d) full context	0.712	n/a

Table 3: consolidation on the first synthetic corpus.

On the first corpus, uniform replay recovers 14 of the 114 evicted facts ($+4.0\sigma$ over zero) and compaction-gated consolidation recovers none. We then generated four more corpora from scratch with different seeds.

corpus	evicted	cascading	compaction	uniform
0	114	0.000	0.000	0.123
1	102	0.000	0.029	0.078
2	112	0.000	0.000	0.009
3	98	0.000	0.020	0.061
4	112	0.000	0.036	0.107
mean		0.000	0.017	0.076

Table 4: fraction of evicted facts recovered on five independently generated corpora.

Is this a fair comparison? Uniform replay trains on dropped spans as well as kept ones, so it can see the text of facts that are later evicted, while the gated arm cannot. That asymmetry is the hypothesis under test, not a flaw in the design. Compaction is cascading: a fact kept at one compaction can be dropped at a later one, and the gate was meant to consolidate such facts while they were still kept. The gated arm does recover some evicted facts on three of the five corpora, so this route exists. The question is whether spending a fixed training budget on the compactor’s selection beats spending it on no selection at all.

Uniform replay beats the compaction gate on all five corpora (Figure 2, left), with a paired $t(4)$ of 3.10 against a critical value of 2.78. The first corpus happened to be the most favourable; after three

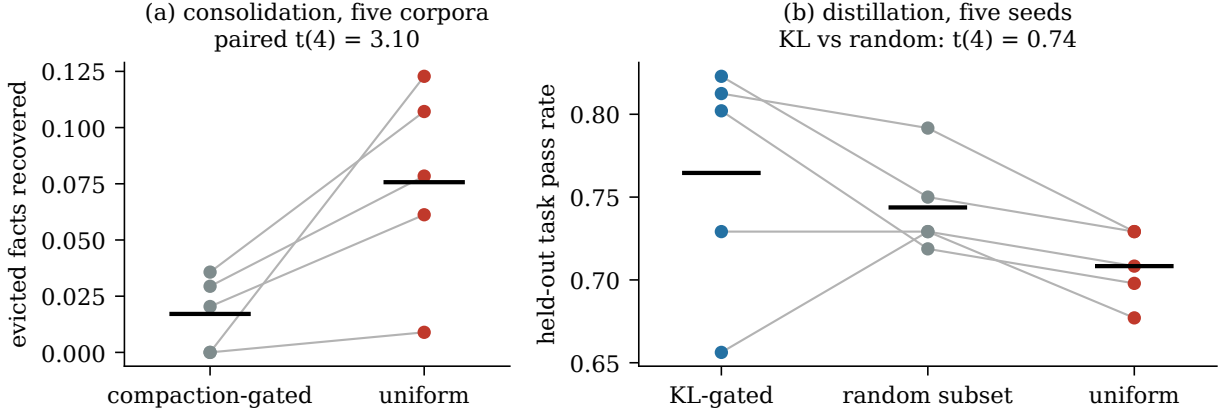


Figure 2: Left: evicted facts recovered by compaction-gated and uniform consolidation on each of five corpora; grey lines join the same corpus and black bars are means. Two corpora have compaction at exactly zero, so their points overlap. Right: held-out pass rate for KL-gated, random-subset and uniform distillation over five seeds, with floor weight 0.1.

corpora the margin was not yet significant ($t(2) = 1.81$). The advantage is confined to evicted facts, and overall accuracy is level ($t(4) = -0.88$).

The cause is coverage. Both adapters fit their training data almost perfectly, with validation cross-entropy of 0.0003 for ours and 0.0005 for uniform. What differs is the training data. Because the compactor never keeps `auth_header` and almost never keeps `config_flag` (§4.1), the gated adapter never sees those facts, while uniform sampling does. Whatever the compactor systematically leaves out, the adapter never learns.

Mixing dropped spans back in does not fix it. Swapping 25% or 50% of the gated budget for randomly chosen dropped spans, with the total budget unchanged, recovers 2 and 1 of the 114 evicted facts. The dropped pool is mostly filler (5,956 filler spans), so random draws rarely hit the missing facts. Drawing round-robin across fact types instead roughly doubles recovery at 25% (4 of 114) but drops to zero at 50%. These are single runs, and all of them fall far short of uniform’s 14. Uniform replay’s advantage seems to come from covering every span over 25 phases rather than from any one-off adjustment to the mix.

The same mask works for abstention. Following Khodabandehlou & Krishnamachari (2026), we trained a LoRA on the same free label to refuse when the evidence had been evicted. It abstains on 0.728 of evicted probes, against 0.000 for the base model ($+17.5\sigma$), which cuts hallucination on those probes by 72.8%. It refuses on only 10.9% of probes whose answer is still in context, and it raises accuracy on retained probes from 0.684 to 0.770. The compaction mask tells a model what it has lost. That is enough to know when to abstain, but not enough to get the facts back.

Natural prose. On HotpotQA (one seed, 160 probes, 92 of them evicted) the methods do not separate. Cascading with no adapter recovers 4 of 92, compaction 3 and uniform 5, a gap of $+0.7\sigma$ between uniform and compaction. Some answers survive elsewhere in the context or in the model’s prior knowledge, so the floor is above zero. A run of this size can neither confirm nor contradict the synthetic result.

5.2 Gating distillation on the context gap

Scoring. For a skill document d , a demonstration query q and its response r , we build $T = \text{chat}(\text{system}, d \text{ ++ } q)$ and $S = \text{chat}(\text{system}, q)$, run the frozen backbone on $T \text{ ++ } r$ and $S \text{ ++ } r$, and score each response token by

$$\text{kl_i} = \text{KL}(p(\cdot \mid T, r_{<i}) \parallel p(\cdot \mid S, r_{<i}))$$

We group tokens into sentence and code-line spans using the tokenizer’s offset mapping, and score each span by the mean over its tokens.

Gating and loss. We give the top 25% of spans weight 1 and the rest a floor weight. The teacher is the backbone with the adapter switched off and the document in context; the student is the model with the adapter switched on and no document. The loss is $\sum \mathbf{w}_i \text{KL}_i / \sum \mathbf{w}_i$. We hold only one model in memory, with one adapter per skill category.

Setup. We wrote eight fictional tool-use APIs in three categories, each with a `SKILL.md`, nine demonstrations and twelve held-out tasks, 96 tasks in all. Because the APIs are invented, the base model cannot already know them. We train Qwen2.5-1.5B-Instruct for 300 steps at a learning rate of 1e-4. A task passes when every required API string appears in the output. The arms are **uniform**, which trains on every token at full weight in the style of Zhang & Qi (2026); **random**, which gives full weight to a random 25% of spans and the floor weight to the rest; and **kl_top**, which does the same with the 25% selected by KL.

The gate carries a signal. Required-token coverage is the share of each demonstration’s required API identifiers that fall inside the selected spans. For the gate it is 0.597, against 0.392 for a random selection of the same size (+6.77 σ). At token granularity the figures are 0.363 and 0.247 (+4.29 σ).

It does not improve training. With the floor weight at 0, **kl_top** passes 0.510 of tasks, below **random** (0.583) and **uniform** (0.708). The tokens leading up to each selected identifier get no gradient at all. A floor weight of 0.1 fixes this.

seed	kl_top	random	uniform
0	0.729	0.729	0.708
1	0.823	0.750	0.729
2	0.802	0.719	0.698
3	0.813	0.792	0.729
4	0.656	0.729	0.677
mean	0.765	0.744	0.708

Table 5: held-out pass rate with floor weight 0.1.

Since **kl_top** and **random** use exactly the same masking, the difference between them measures the ranking alone. It is +0.021 with $t(4) = 0.74$, and its sign changes from seed to seed (Figure 2, right), so the KL ranking adds nothing over a random choice. Both masked arms do beat full-weight **uniform** (**random** by 0.035, $t(4) = 3.90$; **kl_top** by 0.056, $t(4) = 2.33$, which is not significant). This is a separate effect of masked versus full-weight training, and it does not appear to come from poor tuning of **uniform**: its pass rate falls steadily from 300 to 1,200 steps (0.708 to 0.656), and the best of five learning rates between 5e-5 and 8e-4 reaches 0.750 (one seed each). At token granularity with floor 0.1, **kl_top** and **random** are again level (0.771 vs 0.760). The distilled skill also keeps some of its function when retrieval supplies the wrong document, passing 0.292 of tasks against 0.000 for the prompted skill.

5.3 Two failures with different causes

The compaction gate does harm, because its selection leaves out entire fact types and none of the remixing strategies we tried puts them back. The context-gap gate does neither harm nor good. It picks out informative tokens, but training on them works no better than training on a random subset of the same size. Both signals clear their matched controls, and neither beats uncured coverage. A signal’s strength tells us that it carries information. It does not tell us whether that information is what the weights are missing, and we had to test that separately, by comparing the gated training against uniform sampling at the same budget.

6 Matched controls, and what they caught

We report every signal-strength figure against a matched control because, over the course of this project, controls repeatedly caught errors we had not noticed.

Artifact	Apparent effect	Caught by
Silent fallback to a rule-based compactor	3.99x lift	decision-provenance counter
“Keep at most N” answered NONE on every event	100% empty keeps	empty-keep rate
Model answered with the first N spans	1.68x lift	positional control
<code>sorted(kept)[:budget]</code> reimposed position after shuffling	1.69x lift	kept-index distribution
Supporting paragraphs stacked at the front of a benchmark	1.76x lift	positional control (2.44x)
Cached scores from an older tokenizer inverted the gate	-5.14σ	re-scoring in the current environment ($+6.40\sigma$)
Falling CE on gold answers read as knowledge transfer	$6.29 \rightarrow 2.24$	distractor ranking (at chance)
Eval questions that did not identify which item was meant	50% ceiling	duplicate-answer audit

Table 6: artifacts caught by controls.

Six of the eight made a result look better than it was, and two hid a result that might have been positive. None of them was a modelling mistake. All were in how the experiments were built.

The tokenizer case is worth describing because it went against us, which meant scepticism did not catch it, and because we first got the diagnosis wrong. The gate had been scored from a cached file produced under an older tokenizer, which split identifiers such as `vx_stash` into five pieces. Code spans averaged low as a result, and the gate reached only 0.118 required-token coverage against 0.307 for its control. We put this down to mean-pooling. When we re-scored in the current environment, without changing any code, the figures were 0.586 against 0.392. Having seen the stale number, we had also marked the downstream comparison as confounded, but re-running it reproduced the original result exactly. Cached scores are only valid in the environment that produced them, and withdrawing a result calls for as much evidence as reporting one.

The last two rows interact. Consolidation lowered the per-token cross-entropy of evicted gold answers from 6.29 to 2.24, which we first took as evidence that the knowledge had reached the weights. Ranking each gold answer against distractors from the same fact bank then put every method at the chance rate of 0.240, which we took as evidence that it had not. Both readings were wrong. In the original benchmark, 98.3% of questions had more than one correct answer across trajectories: “Which header carries the auth token?” was asked of 48 conversations with four different answers. A model that remembered every trajectory perfectly still could not have scored above 50%, and the distractors were other trajectories’ correct answers. Tying every question to its own trajectory is what made the results in §5.1 measurable. The check that found the problem, counting how many distinct answers each question string has, takes a few minutes and should come before an experiment rather than after it.

7 Discussion

For anyone building free-supervision pipelines with small models, our results suggest two checks. The first is to take the signal from what the model does rather than from what it says. The context gap satisfies this by construction, the compaction decision satisfies it once it is read from the logits rather than generated, and Declarative Attention as published does not, which is what §4.2 measures. The second is to compare

training on the signal against uniform sampling before relying on it. Both consolidation signals fail this second check.

The two checks are related. The measured compaction decision is a strong signal because it reflects what the compactor considers important, and that same property makes it a poor training target, since what a compactor keeps for immediate use is not necessarily what the weights are missing. A signal can be highly informative about one decision and biased for another. This suggests looking for consolidation signals whose errors are uncorrelated with coverage, or using free signals for the decisions they were produced for, as the abstention result does.

For routing, the logit read is a cheap substitute for declaration that needs no training and works at scales where declaring does not. It is not a usable router on its own: at roughly three times chance, most probes are still sent to the wrong region, and it costs K prompts rather than one generated token. The obvious next test is whether it keeps its lead at 27B and above, where declaration works.

8 Limitations

- **Synthetic consolidation.** Our significant consolidation result comes from a controlled synthetic benchmark. The HotpotQA run has one seed, 92 evicted probes and a floor above zero, and is too small to detect a gap of the size seen on synthetic data. The unmarked variant of the synthetic generator is a floor case, since facts and filler come from the same template bank.
- **Seeds.** The main comparisons use five seeds (§5.1 corpora and the §5.2 gate) or three (§4.2). The remixing ablations, abstention, the step and learning-rate sweeps and HotpotQA are single runs.
- **Scale.** Everything is at or below 7B, and we had to quantise the 7B model to 4 bits. We cannot test the 27B+ regime studied in Ho et al. (2026), so our results bound the generated declaration from below (at a fixed-slot control at 1.5B, under half the logit read’s signal at 7B) and do not contradict the accuracy reported there. We have not tested the logit read at that scale.
- **Routing accuracy.** The best router is about three times chance. §4.2 compares ways of eliciting a routing decision; it does not produce a usable router.
- **One model family.** Every result is on Qwen2.5.
- **Multiple comparisons.** We report about twenty paired tests without correcting for multiplicity. Under a Bonferroni correction (critical $t(2) \approx 20$, $t(4) \approx 6.8$) only the largest margins survive: **read** over **generate** in hit rate at 1.5B and 7B, and **read** over **attention_late** in hit rate at 0.5B. The 7B lead of **read** over **attention_late** and the consolidation margin ($t(4) = 3.10$) would not. We treat those as the pre-specified main comparisons of each experiment, but a reader who wants family-wise control should read them as suggestive.
- **Marked synthetic data.** The consolidation benchmark marks facts with a phrase the 1.5B compactor detects, which inflates its salience lift (§4.1). This makes the gate’s loss to uniform replay more striking rather than less, but it means the $+15.85\sigma$ figure should not be read as the compactor’s salience on natural text.

9 Conclusion

Free salience signals do exist in small models, but mostly when they are measured. Asked to state what matters, models between 0.5B and 7B either fail or convey only a fraction of what their logits contain. And a measured signal that clears its control can still be the wrong thing to train on: compaction-gated consolidation lost to uniform replay on every corpus, and context-gap gating did no better than random selection. In both cases the problem only became visible because a matched control sat next to the number, and we think that practice should extend to the decision of what to train on.

10 Broader impact statement

This work studies how small language models can decide what to keep from their context, either by writing it into their weights or by choosing what to read. Consolidating an agent’s history into its weights raises a

privacy concern that retrieval does not: anything in the context, including personal or confidential information, can end up stored in the adapter and is harder to inspect or delete than a document in a store. Our experiments use synthetic conversations, invented APIs and public HotpotQA data, so no personal data was involved, but anyone deploying consolidation on real user histories should filter what reaches training. On the positive side, the abstention result in §5.1 suggests that the same free signal can make a model refuse rather than guess when its evidence has been evicted, which reduces hallucination. We do not see a direct route to misuse beyond that of the underlying models.

References

- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv:2501.00663, 2024.
- Ashwin Gerard Colaco and Nada Lahjouji. What to keep, what to forget: A rate–distortion view of memory compaction in LLMs and agents. arXiv:2607.08032, 2026.
- Simon Dennis, Kevin Shabahang, Hao Guo, and Rivaan Patil. Beyond inference-only deployment: Comparing weight-based consolidation against cascading compaction. arXiv:2605.24657, 2026.
- Yuchen Guo, Junli Gong, Weicheng Wang, Hongmin Cai, Yiu-ming Cheung, and Weifeng Su. PEAM: Parametric embodied agent memory through contrastive internalization of experience in Minecraft. arXiv:2605.27762, 2026.
- Namgyu Ho, Huzama Ahmad, Woosung Koh, Se-Young Yun, Tal Schuster, and Cicero Nogueira dos Santos. Language models can control their own attention. arXiv:2609.02737, 2026.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.
- Mohammadali Khodabandehlou and Bhaskar Krishnamachari. Compression-aware abstention: Teaching LLMs to refuse when KV-compression masks remove answer evidence. arXiv:2608.29934, 2026.
- Yujia Li, Zhenyu Hou, Yi Jing, Jie Tang, and Yuxiao Dong. CompactionRL: Reinforcement learning with context compaction for long-horizon agents. arXiv:2607.05378, 2026.
- Zhenghao Lin et al. Rho-1: Not all tokens are what you need. arXiv:2404.07965, 2024.
- Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.
- Dhruv Saini and Rohan Pandey. ThinkSwitch: Context distillation with LoRA and weight interpolation for specific-purpose reasoning tasks. arXiv:2606.01080, 2026.
- Zekun Wang, Anant Gupta, Zihan Dong, and Christopher J. MacLellan. Self-consolidating language models: Continual knowledge incorporation from context. arXiv:2605.07076, 2026.
- Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality. arXiv:2404.15574, 2024.
- Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token importance in on-policy distillation. arXiv:2604.14084, 2026.
- Tianyi Zhang and Zhonghao Qi. Skill-to-LoRA: From using skills to learning behaviors for token-efficient LLM agents. arXiv:2606.16769, 2026.
- Yong Zhang, Heng Li, Yanwen Huang, Ning Cheng, Yang Guo, Yun Zhu, et al. Sentinel: Decoding context utilization via attention probing for efficient LLM context compression. arXiv:2505.23277, 2025.

A Hyperparameters

	§5.1 consolidation	§5.2 distillation	§4.2 routing
backbone	Qwen2.5-0.5B (compactor 1.5B)	Qwen2.5-1.5B	Qwen2.5-0.5B / 1.5B / 7B
LoRA	r 16, α 32, dropout 0.05, all projections	same	n/a
learning rate	5e-5	1e-4	n/a
steps	80 per sleep phase	300	n/a
schedule	sleep every 4 compaction events, 25 phases	one adapter per category	n/a
replay	reservoir, capacity 80	n/a	n/a
selection	keep $\rho = 0.25$, sentence spans, 8 recent turns protected	top 25% of spans, floor 0 or 0.1	K = 8 regions
data	48 trajectories $\times$ 120 turns $\times$ 6 facts	8 APIs, 9 demos, 12 tasks each	96 HotpotQA trajectories $\times$ 4 probes
seeds	5 corpora	5	3 layouts
precision	fp16	fp16	fp16; 7B nf4

B Reproducibility

The code, configuration files and Kaggle notebooks are in the supplementary repository. Each result maps to one configuration and one notebook. `docs/project_a_findings.md`, `docs/project_b_findings.md` and `docs/declarative.md` contain the per-run tables, `eval/declare_report.py` regenerates Table 2 and its paired tests from the per-run summaries in `results/declare/`, and `paper/figures/make_figures.py` regenerates both figures from `paper/figures/data.json`. The 7B routing runs use `configs/kaggle_declare_7b.yaml` with a repeat-KV SDPA attention (`declare/elicit.py`). It gives the same outputs as stock SDPA but avoids building a 5 GB attention matrix for 9k-token prompts on a T4.

Compute. Every experiment ran on a single NVIDIA T4 (16 GB) through Kaggle, one or two sessions at a time. The total is roughly 50 GPU-hours, of which about 20 went to the 7B routing runs and most of the rest to the consolidation corpora and the distillation seeds.

Assets and licences. Qwen2.5-0.5B, 1.5B and 7B-Instruct are released under the Apache 2.0 licence, and HotpotQA under CC BY-SA 4.0. The synthetic trajectories and the eight tool-use APIs were written for this work and are included in the supplementary repository.